LAB
11

Operating Systems · FCIM / FAF · UTM · 2026-2027

The Race and the Semaphore

A **Track A: the real OS, from the outside (Linux) · 90 minutes**

AT A GLANCE

Part 1	See the race	~15 min
Part 2	Fix it: mutex, then semaphore	~25 min
Part 3	A semaphore as a signal	~20 min
Part 4	A semaphore as a counter: the parking lot	~20 min

Goal

Two threads that touch the same variable at the same time lose work. You see it happen, then you fix it with a mutex and with a semaphore, and you use the same semaphore in three ways: as a lock, as a signal and as a counter. Lab 12 continues with the tools working together.

Before you start

- Work on your own Linux laptop, in a new folder: `mkdir ~/os-lab11 && cd ~/os-lab11`.
- Every program is complete. Type it (or copy it), compile it with -pthread, run it. Compile WITHOUT -O2 today: the optimizer would hide the race of Part 1.
- Keep a file report.md open: each "Observe" box asks for short answers there.

Minimum to pass

Parts 1, 2 and 3 run, with their Observe answers. Part 4 brings the rest of the grade.

PART 1 — See the race ~15 min

Two threads each add 1 to the same counter, one million times. The right answer is 2 000 000.

FILE race.c

```
/* race.c - two threads add to the same counter, with no protection.
 * Build:  gcc -Wall -pthread -o race race.c      (no -O2: keep counter++ in memory)
 * Run:    ./race   (several times)
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;                 /* shared by both threads */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;                /* read, add 1, write back: three steps */
    return NULL;
}
```

```
int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o race race.c
$ ./race
$ ./race
$ ./race
```

Real output (your numbers will be different, but never 2 000 000):

TERMINAL

```
counter = 1050574 (expected 2000000)
counter = 1096707 (expected 2000000)
counter = 1146730 (expected 2000000)
```

Observe (write in report.md)

1. counter++ looks like one step. Write the three steps the CPU really does.
2. Describe, with those steps, how two threads can lose one addition.
3. Why is the result different at every run?

PART 2 Fix it: mutex, then semaphore ~25 min

2.1 A mutex around counter++

The line between lock and unlock is the critical section: only one thread may be inside.

FILE mutex.c

```
/* mutex.c - the same counter, now protected by a mutex.
 * Build:  gcc -Wall -pthread -o mutex mutex.c
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);      /* only one thread past this line */
        counter++;                      /* the critical section */
        pthread_mutex_unlock(&lock);    /* let the other one in */
    }
    return NULL;
```

```
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o mutex mutex.c
$ ./mutex
```

Expected output:

EXPECTED OUTPUT

```
counter = 2000000 (expected 2000000)
```

2.2 The same lock, made from a semaphore

A semaphore is a number that never goes below 0. sem_wait takes one unit (and waits if there is none), sem_post gives one back. Start it at 1 and it behaves like a mutex:

FILE sem.c

```
/* sem.c - the same counter, protected by a semaphore that starts at 1.
 * Build:  gcc -Wall -pthread -o sem sem.c
 */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 1000000

long counter = 0;
sem_t s;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s);           /* 1 -> 0: I go in; at 0: wait */
        counter++;
        sem_post(&s);           /* 0 -> 1: the next one may go in */
    }
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 1);         /* 0 = shared between threads, 1 = start value */
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    sem_destroy(&s);
    return 0;
```

```
}
```

TERMINAL

```
$ gcc -Wall -pthread -o sem sem.c
$ ./sem
```

Expected output:

EXPECTED OUTPUT

```
counter = 2000000 (expected 2000000)
```

2.3 The price of the lock

TERMINAL

```
$ time ./race
$ time ./mutex
$ time ./sem
```

On the reference machine: race 0.00 s (and wrong), mutex 0.10 s, sem 0.12 s. Yours will differ, the order will not.

Observe (write in report.md)

1. Run mutex and sem three times each. Is the answer always 2 000 000?
2. What would happen in sem.c with sem_init(&s, 0, 2)? Try it and explain.
3. Why is the locked version slower?

PART 3 A semaphore as a signal ~20 min

Now the problem is not two threads at once, but the ORDER: B must read the message only after A has written it. A semaphore that starts at 0 is a closed door: B waits at sem_wait until A calls sem_post.

FILE order.c

```
/* order.c - thread B must wait until thread A has written the message.
 * A semaphore that starts at 0 is a signal: sem_wait blocks until someone posts.
 * Build:  gcc -Wall -pthread -o order order.c
 */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

char message[32];
sem_t ready;                    /* starts at 0: nothing is ready yet */

void *writer(void *arg) {
    sleep(1);                   /* pretend the work takes a while */
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready);           /* 0 -> 1: wake up B */
    return NULL;
}
```

```
void *reader(void *arg) {
    printf("B: waiting for the message...\n");
    sem_wait(&ready);           /* blocks while the value is 0 */
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);   /* B starts first, on purpose */
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o order order.c
$ ./order
```

Expected output:

EXPECTED OUTPUT

```
B: waiting for the message...
A: message written
B: I read "hello"
```

B prints its first line at once, then nothing happens for one second, then A and B finish.

Observe (write in report.md)

1. Change the start value to 1 (sem_init(&ready, 0, 1)), compile, run. What does B read now, and why?
2. Put it back to 0. In one sentence: what does a semaphore that starts at 0 mean?

PART 4 A semaphore as a counter: the parking lot ~20 min

Six cars, three places. The semaphore starts at 3 and counts the free places. The mutex is there only so that the variable inside and the printf stay together.

FILE parking.c

```
/* parking.c - 6 cars, 3 places. A semaphore that starts at 3 counts the free places.
 * Build:  gcc -Wall -pthread -o parking parking.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define PLACES 3
#define CARS   6

sem_t places;                                   /* free places */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

```
int inside = 0;                                 /* only for printing */

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places);                          /* take a place, or wait */
    pthread_mutex_lock(&lock);
    inside++;
    printf("car %ld enters  (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);

    sleep(1);                                   /* parked */

    pthread_mutex_lock(&lock);
    inside--;
    printf("car %ld leaves  (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sem_post(&places);                          /* give the place back */
    return NULL;
}

int main(void) {
    sem_init(&places, 0, PLACES);
    pthread_t t[CARS];
    for (long i = 0; i < CARS; i++)
        pthread_create(&t[i], NULL, car, (void *)(i + 1));
    for (int i = 0; i < CARS; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&places);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o parking parking.c
$ ./parking
```

Real output (the order of the cars can differ):

TERMINAL

```
car 1 enters  (inside: 1)
car 2 enters  (inside: 2)
car 3 enters  (inside: 3)
car 1 leaves  (inside: 2)
car 2 leaves  (inside: 1)
car 4 enters  (inside: 2)
car 5 enters  (inside: 3)
car 3 leaves  (inside: 2)
car 6 enters  (inside: 3)
car 4 leaves  (inside: 2)
car 5 leaves  (inside: 1)
car 6 leaves  (inside: 0)
```

Observe (write in report.md)

1. What is the largest value of "inside" you see? Which line of code guarantees it?
2. Set PLACES to 1. What does the semaphore become?

Optional (not graded): no lock at all

For a single counter there is a third way: an atomic operation. atomic_fetch_add does load, add and store as ONE indivisible step of the CPU, so no thread can cut in. Nothing to install: stdatomic.h comes with gcc.

FILE atomic.c

```
/* atomic.c - the counter of race.c, with no lock at all: one atomic instruction per
addition.
 * Build:  gcc -Wall -pthread -o atomic atomic.c
 */
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>

#define N 1000000

atomic_long counter = 0;          /* every operation on it is indivisible */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        atomic_fetch_add(&counter, 1);   /* load, add, store as ONE step */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", (long)counter, 2 * N);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -pthread -o atomic atomic.c
$ time ./atomic
```

Real output (0.03 s on the reference machine, against 0.10 s for the mutex):

TERMINAL

```
counter = 2000000 (expected 2000000)
real 0.03s
```

What you hand in

- race.c, mutex.c, sem.c, order.c, parking.c
- report.md with the Observe answers of Parts 1 to 4.

Submit by git: push to your course repository; the assistant pulls it.

Criterion	Points
Part 1: race seen and explained	**2 p**
Part 2: mutex and semaphore give 2 000 000, Observe answered	**3 p**
Part 3: order with a semaphore at 0, Observe answered	**3 p**
Part 4: parking never above 3, Observe answered	**2 p**
Total	**10**